1.1 What Are Navigation Properties?

Navigation Properties are properties in Entity Framework Core that allow developers to navigate between related entities using object-oriented code instead of manually writing SQL JOIN statements.

For example, a Student belongs to a Department. The Student entity can contain a Department navigation property.

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public int DepartmentId { get; set; }
    public Department Department { get; set; } = null!;
}
```

The Department entity can contain a collection of students:

```
public class Department
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public ICollection<Student> Students { get; set; } = new List<Student>();
}
```

Here, DepartmentId is the Foreign Key, while Department and Students are Navigation Properties.

1.2 How Do Navigation Properties Work?

Entity Framework Core uses the relationships configured in the model to connect related entities. Navigation Properties allow developers to access related objects naturally.

```
var student = context.Students
    .Include(s => s.Department)
    .FirstOrDefault(s => s.Id == 1);

Console.WriteLine(student?.Department.Name);
```

The Include() method requests the related Department data as part of the query. Without loading the related entity, the navigation property may not contain the related data, depending on the loading strategy.

1.3 Two Practical Use Cases

Use Case 1: Displaying a Student's Department

In a university management system, each student belongs to a department. A Navigation Property allows the application to display the student's name and department name together.

```
var students = context.Students
    .Include(s => s.Department)
    .ToList();
```

Use Case 2: Displaying Course Results

A student can enroll in multiple courses, and each course can have multiple students. A junction entity such as StuCrsRes can store the Student ID, Course ID, and Grade.

```
public class StuCrsRes
{
    public int StudentId { get; set; }
    public int CourseId { get; set; }
```

```
    public int Grade { get; set; }
    public Student Student { get; set; } = null!;
    public Course Course { get; set; } = null!;
}
```

Navigation Properties allow the application to access the student and course associated with each result.

1.4 Before Entity Framework Core vs. After Entity Framework Core

Before using an ORM such as Entity Framework, developers commonly retrieved related data using SQL JOIN statements and manually mapped database rows to application objects.

```
SELECT Students.Name, Departments.Name AS DepartmentName
FROM Students
INNER JOIN Departments
    ON Students.DepartmentId = Departments.Id;
```

With Entity Framework Core, developers can express relationships through entity classes and query related data using LINQ and Navigation Properties.

```
var students = context.Students
    .Include(s => s.Department)
    .ToList();
```

Important: SQL JOINs are still used by Entity Framework Core when appropriate. EF Core generates SQL queries from LINQ expressions; it does not eliminate SQL.

2. Loading Related Data in Entity Framework Core

Entity Framework Core provides three common strategies for loading related data: Eager Loading, Explicit Loading, and Lazy Loading.

2.1 Explicit Loading

Definition

Explicit Loading means retrieving related data manually after the main entity has already been loaded. The developer decides when to load the related entity or collection.

Example: Reference Navigation

```
var student = context.Students
    .FirstOrDefault(s => s.Id == 1);

if (student != null)
{
    context.Entry(student)
        .Reference(s => s.Department)
        .Load();
}
```

For a collection navigation property, developers can use Collection():

```
var department = context.Departments
    .FirstOrDefault(d => d.Id == 1);

if (department != null)
{
    context.Entry(department)
```

```
        .Collection(d => d.Students)
        .Load();
}
```

Advantages

- Gives the developer control over when related data is retrieved.
- Avoids loading related data that is not needed.
- Useful when the decision to load related data depends on application logic.

Disadvantages

- Requires additional code.
- Can cause multiple database round trips.
- Repeated loading inside loops can lead to performance problems.

2.2 The N+1 Query Problem

The N+1 Query Problem occurs when an application executes one query to retrieve a collection of entities and then executes an additional query for each entity to retrieve related data.

For example, if one query retrieves 100 students and the application loads each student's department separately, it may execute one query for students plus up to 100 additional queries. The exact number depends on relationship configuration, tracking, and whether related entities are already available.

```
var students = context.Students.ToList();

foreach (var student in students)
{
    context.Entry(student)
        .Reference(s => s.Department)
        .Load();

    Console.WriteLine(student.Department.Name);
}
```

How to Avoid the N+1 Problem

```
var students = context.Students
    .Include(s => s.Department)
    .ToList();
```

Other approaches include projecting only required fields with Select() or loading related data in batches.

2.3 Eager Loading

Definition

Eager Loading retrieves the main entities and their related data as part of the original query operation. It is commonly implemented using Include() and ThenInclude().

Example Using Include()

```
var students = context.Students
    .Include(s => s.Department)
    .ToList();
```

This requests students and their associated departments together. Depending on query configuration, EF Core may generate SQL JOINs or use multiple queries.

Example Using ThenInclude()

```
var departments = context.Departments
    .Include(d => d.Students)
        .ThenInclude(s => s.CourseResults)
    .ToList();
```

ThenInclude() loads a related entity or collection through a previously included navigation property. This example assumes a CourseResults navigation property exists on Student.

Advantages

- Makes required related data available immediately.
- Can reduce repeated database round trips.
- Useful for pages that display several related entities together.

Disadvantages

- Can retrieve more data than necessary.
- Loading multiple collections may create large result sets or duplicate data in SQL JOIN results.
- Complex queries may affect performance.

When to Use Eager Loading

Use Eager Loading when the application already knows that related data will be needed. For example, a student details page may require the student's Department information.

2.4 Lazy Loading

Definition

Lazy Loading retrieves related data automatically when a navigation property is accessed, provided Lazy Loading is configured and supported for that entity. A common EF Core approach uses the Proxies package.

Configuration

```
Install-Package Microsoft.EntityFrameworkCore.Proxies -Version 8.0.0

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"))
    .UseLazyLoadingProxies());
```

Proxy-based Lazy Loading generally requires navigation properties to be virtual and entity classes to meet the proxy requirements.

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public int DepartmentId { get; set; }
    public virtual Department Department { get; set; } = null!;
}
```

Example

```
var student = context.Students
    .FirstOrDefault(s => s.Id == 1);

if (student != null)
{
    Console.WriteLine(student.Department.Name);
}
```

If Lazy Loading is configured correctly and the entity is tracked, accessing student.Department can trigger a database query automatically.

Advantages

- Related data is retrieved only when accessed.
- Can simplify code when navigation properties are used selectively.
- May avoid loading unnecessary related data.

Disadvantages

- Queries can occur unexpectedly when properties are accessed.
- Can cause the N+1 Query Problem.
- May create performance problems in loops or API serialization.
- Requires appropriate configuration and entity requirements.

When to Use Lazy Loading

Lazy Loading may be useful when related data is accessed occasionally and only a small amount of related information is needed. For APIs and performance-sensitive applications, explicit query control is often easier.

2.5 Comparison of Loading Strategies

Feature	Eager Loading	Explicit Loading	Lazy Loading
When data loads	As part of the initial query	When explicitly requested	When navigation property is accessed
Developer control	Chooses what to include	High control over timing	Less visible control over timing
N+1 risk	Lower when used appropriately	Possible in loops	Possible when many properties are accessed
Typical use	Details pages with known related data	Conditional loading	Selective on-demand access

No single loading strategy is always best. The choice depends on required data, query performance, database round trips, and application complexity.

3. Builder Design Pattern

3.1 What Is the Builder Design Pattern?

The Builder Design Pattern is a creational design pattern that separates the construction of a complex object from its final representation. It allows developers to construct objects step by step instead of using a constructor with many parameters.

3.2 What Problem Does It Solve?

A class with many constructor parameters can be difficult to read and maintain, especially when several parameters are optional. A Builder provides a more readable way to configure an object.

```
public class Computer
{
    public string CPU { get; set; } = string.Empty;
    public int RAM { get; set; }
    public int Storage { get; set; }
    public bool HasGraphicsCard { get; set; }
    public bool HasWiFi { get; set; }
}
```

3.3 Main Components

- Product: The complex object being constructed.
- Builder: Defines the construction operations.
- Concrete Builder: Implements the construction steps.
- Director: Optionally controls the order of construction steps.

Not every implementation requires all four components. A fluent builder can use method chaining without a separate Director.

3.4 Practical Example in C#

Product Class

```
public class Computer
{
    public string CPU { get; set; } = string.Empty;
    public int RAM { get; set; }
    public int Storage { get; set; }
    public bool HasGraphicsCard { get; set; }
    public bool HasWiFi { get; set; }
}
```

Builder Class

```
public class ComputerBuilder
{
    private readonly Computer _computer = new Computer();

    public ComputerBuilder SetCPU(string cpu)
    {
        _computer.CPU = cpu;
        return this;
    }

    public ComputerBuilder SetRAM(int ram)
    {
        _computer.RAM = ram;
        return this;
    }

    public ComputerBuilder SetStorage(int storage)
    {
        _computer.Storage = storage;
```

```
        return this;
    }

    public ComputerBuilder AddGraphicsCard()
    {
        _computer.HasGraphicsCard = true;
        return this;
    }

    public ComputerBuilder AddWiFi()
    {
        _computer.HasWiFi = true;
        return this;
    }

    public Computer Build()
    {
        return _computer;
    }
}
```

Using the Builder

```
var computer = new ComputerBuilder()
    .SetCPU("Intel Core i7")
    .SetRAM(32)
    .SetStorage(1000)
    .AddGraphicsCard()
    .AddWiFi()
    .Build();
```

The code constructs the Computer object step by step. Methods return the builder itself, enabling method chaining.

3.5 Two Practical Use Cases

Use Case 1: Building Complex Objects

A computer configuration may contain several optional components. The Builder Pattern lets the application select only the required components while keeping object creation readable.

Use Case 2: Constructing Reports or Documents

A report may contain a title, sections, tables, headers, and footers. A builder can assemble these elements step by step and produce a completed report.

3.6 Advantages

- Improves readability when creating complex objects.
- Supports optional configuration steps.
- Separates construction logic from the product's internal representation.
- Can make construction logic easier to maintain and test.

3.7 Disadvantages

- Adds extra classes and code.
- May be unnecessary for simple objects.
- Can introduce complexity if the object has only a few straightforward properties.

3.8 Applying the Builder Pattern in ASP.NET Core MVC

In an ASP.NET Core MVC application, a builder can construct a complex ViewModel before passing it to a View. For example, a dashboard ViewModel may contain summary statistics, recent student records, and department information. A builder could assemble these components in stages.

However, the Builder Pattern is not required for every MVC application. Simple ViewModels can usually be created directly, and Dependency Injection is a separate concept rather than a replacement for the Builder Pattern.

Final Conclusion

Navigation Properties and data-loading strategies are essential concepts in Entity Framework Core because they define how related entities are accessed and retrieved from a database. Eager Loading, Explicit Loading, and Lazy Loading provide different levels of control over database queries, and understanding the N+1 Query Problem helps developers avoid inefficient data access. The Builder Design Pattern addresses a different concern: creating complex objects in a readable and maintainable way. Together, these concepts improve the design, maintainability, and performance of ASP.NET Core applications when applied appropriately.